

Master's Thesis

Winter Term 2025/2026

Philipp Schlatter

Numerical Studies of Transition and Turbulence in a Flat-Plate Boundary Layer

1 Methods

For this Thesis the high-order Spectral Element (SEM) code NEKO is used. Neko is an open-source computational fluid dynamics (CFD) solver, that is based on the older Nek5000 CFD solver done by ? in the nineties. The portable framework Neko is written in modern Fortran in an object-oriented manner and is mainly focused on incompressible flow simulations ?. Only Direct Numerical Simulation (DNS) will be performed in this work, however Large Eddy Simulation (LES) models are also implemented in NEKO.

1.1 Simulation setup

1.2 Generating rough surfaces

Surfaces in real-world engineering applications often exhibit irregular roughness. In previous studies often scans of real-world surfaces are used, to perform the simulations, e.g. by ?, ? and ?. This yields the advantage, that inherently the real world influence of surface roughness can be investigated. This method is however very restricted by the number of surfaces and hence surface parameters, that can be tested. Also, the investigated surface parameters can not be directly controlled, but are a direct result of the chosen surface to be scanned. In the present work, the surface roughness will be generated by an algorithm introduced by ?. In the following section a brief description of the algorithm is given.

1.2.1 Surface generation algorithm

The idea behind the algorithm by ? is, that a realistic surface topography needs a height distribution, as well as a spacial distribution. To that extend, two main input parameters need to be specified. To define the height distribution, a height probability distribution needs to be specified. To define the spacial distribution, a power spectrum is used. Using the probability distribution and the power spectrum, two initial surfaces are generated. The first surface Z_h shows the desired probability distribution, but not necessarily the power spectrum. The second surface Z_s exhibits the power spectrum but not the probability distribution. The algorithm iteratively corrects Z_s and Z_h to match each other, until a specified convergence is reached. The iteration consists of two steps:

1. Adjust the Z_s using the formula $\hat{Z}_s^{i+1} = \hat{Z}_h^i \frac{|\hat{Z}_s^i|}{|\hat{Z}_h^i|}$, with the values denoted by $\hat{Z}$ representing their respective fourier transforms. The inverse transform of $\hat{Z}_s^{i+1}$, Z_s^{i+1} , shows the specified power spectrum but a changed height probability distribution.
2. Obtain a Z_h^{i+1} by adjusting the height probability distribution of Z_s^{i+1} . This is achieved by rank ordering

In this work, as the random distribution the Weibull distribution is used, with the probability density function

$$f(z) = \frac{k}{\lambda} \left(\frac{z}{\lambda} \right)^{k-1} e^{-(z/\lambda)^k} \quad (1.1)$$

with k being the scale factor and λ representing the scale factor. Z BESCHREIBEN. The used power spectrum is defined by

$$C(q) = \frac{S_q^2 \beta}{2\pi (\beta^2 + q_x^2 + q_y^2)^{\frac{3}{2}}} \quad (1.2)$$

with S_q being the rms height and $\frac{1}{\beta}$ is the autocorrelation length, as described by ?. The specific random distribution and power spectrum are chosen, since they are thoroughly tested by ?. It shows, that reasonably fast convergence, as well as low absolute errors can be achieved. It is important to note, that the two surfaces are not exactly the same, but similar within the bounds of a defined tolerance. Z_h will have the desired height probability distribution, but the power spectrum will deviate slightly, within the capability of the algorithm. Vice versa for Z_s . It is chosen to use the surface Z_h in this work. The described method for generating rough surfaces needs several input parameters, namely k, λ, S_q and β . SCHREIBEN WIE ICH

1.2.2 Applying rough surfaces onto the mesh

The algorithm described in section 1.2.1 yields the height values for a plane surface. This surface has to be mapped onto the pipe in such a way, that the same generated surface can be mapped onto different meshes with varying amounts of grid points. This leads to the fact, that the straightforward method, i.e. applying the height values of the generated surface points directly onto the pipe mesh is not applied in this work. The used approach relies on the fact, that every surface topography can be decomposed into a multitude of sin and cos waves. The decomposed values are the Fourier coefficients, which can be seen as a weight given for each mode. By recombining the extracted modes, the surface can be recreated. The Fourier coefficients are extracted by taking the real and imaginary part of the Fourier transform of my generated rough surface, such that $A(m, n) = \text{Re}(\hat{Z}_h)$ and $B(m, n) = \text{Im}(\hat{Z}_h)$

This decomposition and recreation is only exact, if a infinite number of modes is used. This is however only relevant for regular surfaces. Since the surfaces generated by the algorithm described in section 1.2.1 are defined with a finite amount of modes (that is chosen by the user), this amount of modes is sufficient for the recreation of the surface. With these parameters, the height function for the rough surface at each individual point can be defined as

$$k(th, z) = \sum_{m, n}^{M, N} A(m, n) \cos(phase) - B(m, n) \sin(phase) \quad (1.3)$$

with

$$phase = 2\pi \left(\frac{uz}{L_z} + \frac{vth}{L_{th}} \right) \quad (1.4)$$

with $A(m, n)$ and $B(m, n)$ representing the Fourier coefficients, m and n the Fourier mode and z, th the streamwise and azimuthal coordinate of the respective point. The values u and v represent the shift.... HIER NOCH WAS SCHREIBEN. This height function yield the recreation of the rough surface, which is applied as is at the grid points at the wall. However, a body fitted mesh is desired, which leads to the necessity, to create a scaling function for the grid points away from the wall. The roughness function should be applied onto the mesh in the outer region of the domain, while the inner region stays unaffected HIER VLT CHAN ZITIEREN. The used function in this work is

$$rs = R + \frac{R - r_{crit}}{r_{max} - r_{crit}} * k(th, z) \quad (1.5)$$

with $0 < r_{crit} < 1$ representing the minimal radius, at which the grid points are shifted, r_{max} the maximum radius of the smooth pipe mesh and R the radius of the point to be shifted This function scales the magnitude of shift applied to each grid point linearly between the outer pipe radius of the original smooth pipe and r_{crit} , from one to zero.

The shift applied to the points is only in radial direction. Since the GLL points are distributed onto the mesh before the roughness is applied, the GLL distribution is not exact after the rough deformation. Since the shift is of small magnitude, the effect is assumed to be small. A test on the influence of simulation results is beyond the scope of this work. NOCH AUF ANZAHL DER MODES EINGEHEN -> BEZUG ZU GRIDPOINTS?

GIBTS NE QUELLE FOR DIE HEIGHT FUNCTION?

WIE KOMME ICH AUF DIE FOURIER COEFFICIENTS?

AUF BODY FITTED VS. IBM EINGEHEN

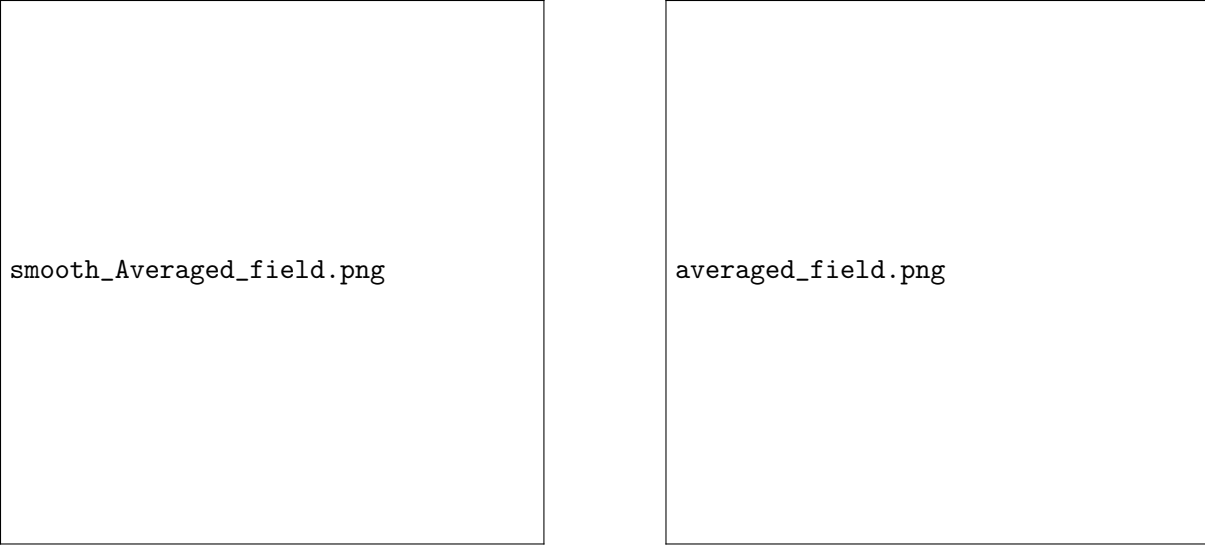
1.2.3 Finding good input parameters

1.3 Postprocessing

In order to be able to work with the obtained data by the DNS, the data has to be postprocessed. In this work, this is mainly done by using PySEMTTools, which is a open source Python package, to postprocess SEM data on hexahedra high-order elements ?.

The typical workflow is as follows:

- In case multiple files exist that contain statistics data, average them to obtain a single averaged field
- Interpolate the averaged data from the SEM mesh to user defined interpolation points



- Calculate the desired values based on the interpolated data
- Further average the data, such that 2D or 1D plots can be generated

To be able to process data of rough cases properly, various changes had to be done compared to a smooth case.

1.3.1 Postprocessing rough cases

The Postprocessing of rough DNS data is not as straightforward as for smooth cases. Two of the several conceivable workflows will be described in this section.

Mapping Interpolation points to match the rough surface

In this approach, the interpolation points are mapped to match the rough surface of the DNS. Due to the matching field, the interpolation points can be interpolated onto the DNS field with all points being found by the interpolator. The main issue with this approach is the fact, that the interpolation points, which contain the data that is used in the further process, are not uniformly distributed in the radial direction. This will lead to the situation, that data points that are interpolated at different radii will be treated as if they were at a comparable position.

Inverse mapping DNS data and interpolation points

The second approach is to introduce a computational domain for the data interpolation. To map the DNS data into the computational domain it is mapped back to a smooth pipe. This is achieved by recalculating the height function of the rough surface using equation 1.3 and equation 1.4. The height function is applied onto the rough mesh using the inverse of the roughness scaling function, equation 1.5. The inverse of the rough surface function can be expressed as

$$surf_{inverse} = \frac{1.0}{(1.0 + \frac{surf_{change}}{pipe_R - r_{crit}})} * (mesh_{change} + \frac{r_{crit}}{pipe_R - r_{crit}} * surf_{change}) \quad (1.6)$$

The results are shown from an exemplary case in figure